EE3621: Digital Signal Processing

Lecture 28: From Adaptive Filtering to Machine Learning — The DSP ↔ ML Dictionary

(III B.Tech EEE)

Comprehensive Lecture Notes & Faculty Textbook Reference

Lecture Plan & Objectives (40 Minutes Breakdown)

- **00:00 – 06:00:** FIR → single-layer ANN / convolution layer; LMS = stochastic gradient descent (ADALINE).
- **06:00 – 13:00:** IIR → RNN: poles as eigenvalues, vanishing/exploding gradients, gated units, state-space models; EMA momentum as an IIR lowpass.
- **13:00 – 20:00:** Fourier transform ↔ eigenvectors: complex exponentials, circulant matrices, PCA/KLT, eigenvalue spread in LMS; noise filtering by SVD.
- **20:00 – 30:00:** Non-local means → BM3D → Transformers (attention); positional encoding as phasors of geometrically spaced frequencies.
- **30:00 – 37:00:** Further analogies (Wiener/regression, Kalman, matched filter, pooling/aliasing, filter banks, residuals, normalization, linear prediction).
- **37:00 – 40:00:** Worked numerical examples, honest limits of each analogy, exam traps.

---

1 1. LEARNING OBJECTIVES

Students will be able to:

1. Map the LMS adaptive FIR filter onto a one-neuron network trained with SGD.
2. Explain why a recurrent layer is a nonlinear IIR filter and relate poles to gradient stability.
3. State why Fourier bases are the eigenvectors of LTI systems and how PCA/KLT generalizes them.
4. Apply truncated-SVD denoising and justify it with the Eckart–Young theorem.
5. Show that non-local-means/BM3D-style patch aggregation is a form of attention.
6. Show that sinusoidal positional encoding is a bank of phasors, and that a position shift is a phase rotation.
7. State where each analogy *breaks down*.

---

2 2. THE MASTER DICTIONARY

DSP / Adaptive filtering	Machine learning	Where the link is exact
FIR filter $y = \mathbf{w}^T\mathbf{x}$	Linear neuron / dense layer; 1-D conv layer	Same equation; a conv layer is a bank of FIR filters with shared taps
LMS (Widrow–Hoff)	SGD on squared loss; delta rule; ADALINE	Identical update $\mathbf{w} \leftarrow \mathbf{w} + \mu e\mathbf{x}$
IIR filter	Recurrent network (RNN, LSTM, SSM)	A linear RNN *is* an IIR filter
Poles inside unit circle	Spectral radius of $\mathbf{W}_h < 1$	Stability $\leftrightarrow$ vanishing/exploding gradients
Fourier basis	Eigenvectors of $\mathbf{R}$, PCA	Complex exponentials diagonalize circulant matrices
SVD/subspace denoising	PCA, low-rank approximation, LoRA	Eckart–Young optimality
BM3D / non-local means	Self-attention, Vision Transformers	NLM weights are Gaussian-kernel softmax attention
Sinusoidal positional encoding	Phasors at frequencies $10000^{-2i/d}$	Shift $\Rightarrow$ rotation (shift theorem)
Wiener filter, $\mathbf{Rw} = \mathbf{p}$	Linear regression normal equations	Same equations
Exponential average	Momentum / Adam moments	First-order IIR lowpass

3 3. FIR FILTER $\rightarrow$ ANN; LMS $\rightarrow$ GRADIENT DESCENT

3.1 3.1 Same equation

FIR / adaptive transversal filter and a single neuron before the activation:

$$y[n] = \sum_{k=0}^{M-1} w_k\, x[n-k] = \mathbf{w}^T\mathbf{x}[n] \qquad \longleftrightarrow \qquad z = \mathbf{w}^T\mathbf{x} + b, \quad a = \varphi(z). \tag{1}$$

The tapped delay line supplies $\mathbf{x}[n]$; in ML the inputs are arbitrary features. Adding a nonlinearity φ gives a perceptron; stacking layers gives a multilayer perceptron (MLP).

3.2 3.2 Convolutional layers

A 1-D convolutional layer slides the *same* FIR filter over the signal (weight sharing = time invariance), uses several filters (channels = a filter bank), applies a nonlinearity, then downsamples. Image CNNs are 2-D FIR filters. Most deep-learning libraries compute cross-correlation, not flipped-kernel convolution, but the difference is only a flip of $\mathbf{w}$.

3.3 3.3 LMS = SGD

With squared loss $\ell = \frac{1}{2}e^2$, $e = d - \mathbf{w}^T\mathbf{x}$, one SGD step is

$$\mathbf{w} \leftarrow \mathbf{w} - \mu\nabla_{\mathbf{w}}\ell = \mathbf{w} + \mu\, e\, \mathbf{x}. \tag{2}$$

This is the LMS update of Lecture 27. Backpropagation is the same rule with the chain rule applied through layers: δ_{layer} plays the role of e.

- **Step size:** μ is the learning rate; the bound $0 < \mu < 2/\lambda_{\max}(\mathbf{R})$ is the quadratic-loss stability limit for gradient descent.
- **Misadjustment vs. speed:** large μ = fast but noisy; the LMS trade-off is the learning-rate/mini-batch trade-off.
- **Convergence modes:** the error along eigenvector $\mathbf{q}_i$ of $\mathbf{R}$ decays as $(1-\mu\lambda_i)^n$. Large *eigenvalue spread* $\lambda_{\max}/\lambda_{\min}$ gives slow convergence — the reason for input whitening/normalization in ML and for RLS/Adam-type preconditioning.
- **Regularization:** leaky LMS ($\mathbf{w} \leftarrow (1-\mu\gamma)\mathbf{w} + \dots$) = weight decay / ridge; diagonal loading of $\mathbf{R}$ = ridge regression.
- **Normalized LMS** $\mu/(\epsilon + \|\mathbf{x}\|^2) \leftrightarrow$ normalized/adaptive learning rates.
- **RLS** $\leftrightarrow$ Newton/second-order methods.

3.4 3.4 Wiener filter = linear regression

The Wiener equations $\mathbf{R}\mathbf{w}^* = \mathbf{p}$ are the normal equations $\mathbf{X}^T\mathbf{X}\mathbf{w} = \mathbf{X}^T\mathbf{d}$ of least squares, with sample averages replacing expectations. Adaptive filtering is online learning; Wiener is the batch solution.

---

4 4. IIR FILTER → RECURRENT NEURAL NETWORK

4.1 4.1 Feedback gives memory

$$\text{IIR: } y[n] = \sum_k b_k x[n-k] - \sum_{k\geq 1} a_k y[n-k] \qquad \text{RNN: } \mathbf{h}_t = \varphi(\mathbf{W}_h\mathbf{h}_{t-1} + \mathbf{W}_x\mathbf{x}_t + \mathbf{b}). \tag{3}$$

Remove the nonlinearity (φ = identity): $\mathbf{h}_t = \mathbf{W}_h\mathbf{h}_{t-1} + \mathbf{W}_x\mathbf{x}_t$, a state-space IIR filter with

$$\mathbf{H}(z) = (\mathbf{I} - \mathbf{W}_h z^{-1})^{-1}\mathbf{W}_x, \qquad \text{poles} = \text{eigenvalues of } \mathbf{W}_h. \tag{4}$$

- **Stability:** BIBO stable iff all eigenvalues of $\mathbf{W}_h$ lie inside the unit circle.
- **Vanishing/exploding gradients:** backprop through time multiplies by $\mathbf{W}_h^T$ repeatedly; the gradient after k steps scales like $\rho(\mathbf{W}_h)^k$. Poles inside → vanish (short memory); outside → explode; near $|z| = 1$ → long memory. This is the impulse response $a^n u[n]$ of a one-pole filter.
- **Nonlinear case:** tanh bounds the state, which prevents blow-up of the forward pass but not of the gradients.

4.2 4.2 Gated RNNs as adaptive-pole IIR filters

A one-pole IIR $y[n] = a\,y[n-1] + (1-a)x[n]$ has fixed leakage a. An LSTM/GRU cell state $c_t = f_t \odot c_{t-1} + i_t \odot g_t$ is the same recursion with a *signal-dependent pole* $f_t \in (0,1)$ (forget gate) and input gain i_t. Modern **state-space models** (S4, Mamba) are explicitly discretized linear systems (zero-order hold/bilinear transform, exactly Lectures 15–27) computed by convolution or recurrence.

4.3 4.3 Worked Example 28.1: EMA momentum is an IIR lowpass

Gradient momentum / exponential moving average: $m_t = \beta m_{t-1} + (1-\beta) g_t$. So $H(z) = \frac{1-\beta}{1-\beta z^{-1}}$ with a single pole at $z = \beta$.

- DC gain: $H(1) = 1$. Gain at $\omega = \pi$: $H(-1) = \frac{1-\beta}{1+\beta}$. For $\beta = 0.9$: $0.1/1.9 \approx 0.053$ ($-25.6\,\text{dB}$).
- Memory: time constant $\tau \approx 1/(1-\beta) = 10$ samples; $-3\,\text{dB}$ frequency $\omega_c \approx 1-\beta = 0.1\,\text{rad}$ (for β near 1).
- Interpretation: momentum is a lowpass filter on the noisy gradient sequence, suppressing high-frequency gradient noise (just like smoothing in DSP).

---

5 5. FOURIER TRANSFORM ↔ EIGENVECTORS

5.1 5.1 Complex exponentials are eigenfunctions of LTI systems

For input $x[n] = e^{j\omega n}$ and impulse response $h[n]$:

$$y[n] = \sum_k h[k] e^{j\omega(n-k)} = H(e^{j\omega})\, e^{j\omega n}. \tag{5}$$

So $e^{j\omega n}$ is an **eigenvector** and $H(e^{j\omega})$ the **eigenvalue**. For finite-length periodic (circular) convolution the operator is a **circulant matrix C**, and

$$\mathbf{C} = \mathbf{F}^H \,\text{diag}\big(H[0], \ldots, H[N-1]\big)\, \mathbf{F}, \tag{6}$$

with $\mathbf{F}$ the DFT matrix: the DFT diagonalizes every circulant matrix. This is why convolution becomes multiplication in the frequency domain.

5.2 5.2 PCA / KLT: data-dependent Fourier

For a zero-mean random vector with covariance $\mathbf{R} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^T$, the Karhunen–Loève transform uses eigenvectors $\mathbf{Q}$ as an optimal decorrelating basis (this is PCA). For a wide-sense stationary signal, $\mathbf{R}$ is Toeplitz and, as the length grows, asymptotically circulant, so the eigenvectors tend to **Fourier vectors** and the eigenvalues to the **power spectral density**:

$$\lambda_i \to S_x(\omega_i). \tag{7}$$

Thus the DFT is the KLT of stationary signals, and PCA is the generalization to non-stationary data.

5.3 5.3 Consequences for adaptive filtering

$\lambda_{\max}/\lambda_{\min} \to S_{\max}/S_{\min}$: a *coloured* input (large spectral dynamic range) slows LMS; white input gives the fastest, equal-rate convergence. Pre-whitening, transform-domain LMS (DFT/DCT-LMS) and feature normalization in ML are all the same fix.

5.4 5.4 Other Fourier ↔ ML links

FFT-based fast convolution; spectrogram/mel inputs to speech models; Fourier features for coordinate networks (sin / cos of random frequencies); Fourier Neural Operators; FNet (token mixing by FFT).

---

6 6. NOISE FILTERING VIA SVD

6.1 6.1 Theory

Any matrix $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T = \sum_i \sigma_i \mathbf{u}_i \mathbf{v}_i^T$. The **Eckart–Young theorem**: the best rank-k approximation in Frobenius (and spectral) norm is the truncated sum $\mathbf{A}_k = \sum_{i \le k} \sigma_i \mathbf{u}_i \mathbf{v}_i^T$.

Signal + white noise: $\mathbf{Y} = \mathbf{S} + \mathbf{N}$ with $\mathbf{S}$ low rank. Noise spreads *evenly* over all singular values (a noise floor $\approx \sigma_n(\sqrt{m} + \sqrt{n})$), while signal energy concentrates in a few large σ_i. Keeping only the large ones removes most noise — a lowpass filter in the *singular spectrum* instead of the frequency spectrum.

6.2 6.2 DSP usage

Take a Hankel/Toeplitz matrix of a 1-D signal (a sum of r sinusoids/damped exponentials gives rank $2r$, or r for complex exponentials); truncating its SVD and anti-diagonal averaging is **Cadzow denoising**. Also: subspace methods (MUSIC, ESPRIT), PCA denoising, singular value thresholding, low-rank adaptation (LoRA) of neural weights.

6.3 6.3 Worked Example 28.2

Noisy observation $\mathbf{Y} = \begin{bmatrix} 3.1 & 2.9 \\ 2.9 & 3.1 \end{bmatrix}$. Eigenvalues of this symmetric matrix: $3.1 \pm 2.9 = 6.0,\ 0.2$ with eigenvectors $\frac{1}{\sqrt{2}}[1,1]^T$ and $\frac{1}{\sqrt{2}}[1,-1]^T$. Singular values: $\sigma_1 = 6.0$, $\sigma_2 = 0.2$. Rank-1 truncation:

$$\mathbf{Y}_1 = 6.0 \cdot \tfrac{1}{2} \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} 3 & 3 \\ 3 & 3 \end{bmatrix}. \tag{8}$$

The rank-one structure (identical rows/columns) is recovered, and the small singular value 0.2 (the noise-like component) is discarded.

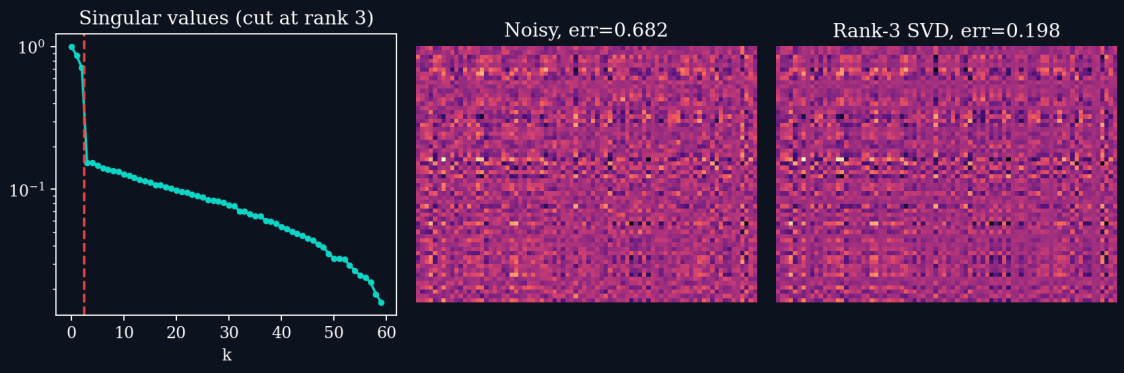


Figure 1: Rank-3 matrix plus white noise: the singular-value spectrum shows a sharp drop after 3; truncation reduces the Frobenius error to the true matrix (error values printed in the panels).

7 7. BM3D → TRANSFORMERS: NON-LOCAL AGGREGATION

7.1 7.1 Non-local means (NLM)

Pixel/sample i is estimated as a weighted average of *all* samples whose surrounding patches look similar:

$$\hat{x}_i = \sum_j w_{ij}\, x_j, \qquad w_{ij} = \frac{\exp\!\big(-\|\mathbf{p}_i - \mathbf{p}_j\|^2/h^2\big)}{\sum_{j'} \exp\!\big(-\|\mathbf{p}_i - \mathbf{p}_{j'}\|^2/h^2\big)}. \tag{9}$$

Compare with scaled dot-product attention

$$\mathrm{Attn}(\mathbf{q}_i) = \sum_j \mathrm{softmax}_j\!\Big(\frac{\mathbf{q}_i^T \mathbf{k}_j}{\sqrt{d}}\Big)\mathbf{v}_j. \tag{10}$$

Since $-\|\mathbf{q}-\mathbf{k}\|^2 = 2\mathbf{q}^T\mathbf{k} - \|\mathbf{q}\|^2 - \|\mathbf{k}\|^2$ and the $\|\mathbf{q}\|^2$ term cancels inside the softmax, Gaussian-kernel NLM equals dot-product attention when key norms are constant (or are absorbed in a bias). So: *patch* = query/key, *block matching* = similarity scores, *weighted aggregation* = softmax-weighted sum of values.

7.2 7.2 BM3D

BM3D (Block-Matching and 3-D filtering) adds three DSP ideas on top of NLM:

1. **Grouping:** find the K patches most similar to a reference patch and stack them into a 3-D array (top-K attention).
2. **Collaborative filtering:** apply a 3-D orthogonal transform (DCT/wavelet in 2-D plus Haar across the stack), *shrink* small coefficients (hard threshold in stage 1, empirical Wiener in stage 2), invert (transform-domain sparsity).
3. **Aggregation:** each pixel gets several estimates from different groups, combined by weighted averaging.

The Transformer block mirrors this: multi-head attention (group + aggregate), then a position-wise MLP (nonlinear "shrinkage" in a learned feature space), with residual connections. Image-restoration Transformers (SwinIR, Restormer) outperform BM3D by *learning* the similarity metric and the transform.

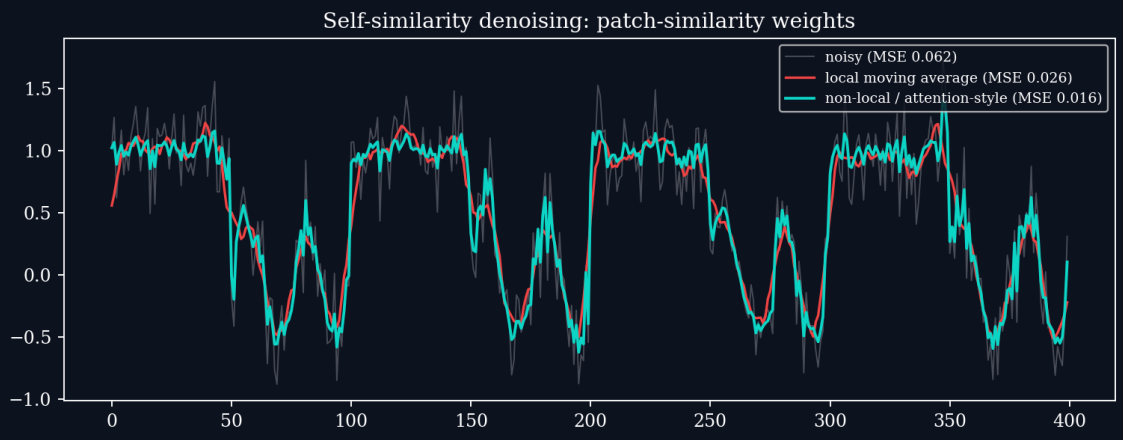


Figure 2: Toy 1-D demonstration: a piecewise signal with repeating structure is denoised either by a local moving average or by similarity-weighted (attention-style) averaging over all positions. Self-similarity lets distant, similar patches reinforce each other without blurring edges. (Parameters here were tuned by hand for illustration only.)

Caveats. BM3D has hand-designed transforms, hard thresholds and a fixed patch metric; attention weights are learned, use projected features $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V$, run several heads and have no explicit threshold. The correspondence is conceptual (non-local self-similarity), not an identity of algorithms.

---

8 8. POSITIONAL ENCODING IS A BANK OF PHASORS

Self-attention is permutation-invariant, so positions must be injected. The original Transformer adds

$$PE(p, 2i) = \sin(\omega_i p), \quad PE(p, 2i+1) = \cos(\omega_i p), \qquad \omega_i = 10000^{-2i/d}. \tag{11}$$

Each pair $(\sin, \cos)$ is the real and imaginary part of a phasor $e^{j\omega_i p}$ rotating with position ("time" is position p). The frequencies are **geometrically spaced** from $\omega_0 = 1$ rad/token down to $\approx 10^{-4}$ rad/token (wavelengths 2π up to $\approx 6 \times 10^4$ tokens) — like a multi-resolution filter bank, or like binary digits of a counter where each bit flips at a different rate.

8.1 8.1 Shift = rotation

$$\begin{bmatrix} \sin \omega(p+k) \\ \cos \omega(p+k) \end{bmatrix} = \underbrace{\begin{bmatrix} \cos \omega k & \sin \omega k \\ -\sin \omega k & \cos \omega k \end{bmatrix}}_{\mathbf{R}(\omega k)} \begin{bmatrix} \sin \omega p \\ \cos \omega p \end{bmatrix}. \tag{12}$$

This is the DFT shift theorem $x[p-k] \leftrightarrow e^{-j\omega k} X$: a position offset k is a *linear map independent of* p, so the network can learn to attend by relative offset. Moreover

$$PE(p)^T PE(p+k) = \sum_i \cos(\omega_i k), \tag{13}$$

which depends only on k — exactly the autocorrelation of the phasor bank.

8.2 8.2 Rotary embeddings (RoPE)

RoPE does not add phasors but *rotates* the query and key by $\mathbf{R}(\omega_i p)$ in each 2-D subspace; then $\langle \mathbf{R}(\omega p)\mathbf{q}, \mathbf{R}(\omega p')\mathbf{k} \rangle = \mathbf{q}^T \mathbf{R}(\omega(p'-p))\mathbf{k}$, so attention depends on relative position only.

8.3 8.3 Worked Example 28.3

Take $d = 8$: $\omega_i = 10000^{-2i/8}$ for $i = 0, 1, 2, 3$: 1, 0.1, 0.01, 0.001 rad/token (wavelengths 6.28, 62.8, 628, 6283 tokens). For $\omega = 1$, $p = 2$, $k = 1$: $PE = [\sin 2, \cos 2] = [0.909, -0.416]$. Applying $\mathbf{R}(1)$ with $\cos 1 = 0.540$, $\sin 1 = 0.841$:

$$\sin 3 = 0.540(0.909) + 0.841(-0.416) = 0.491 - 0.350 = 0.141, \quad \cos 3 = -0.841(0.909) + 0.540(-0.416) = -0.765$$

which matches $\sin 3 = 0.141$, $\cos 3 = -0.990$.

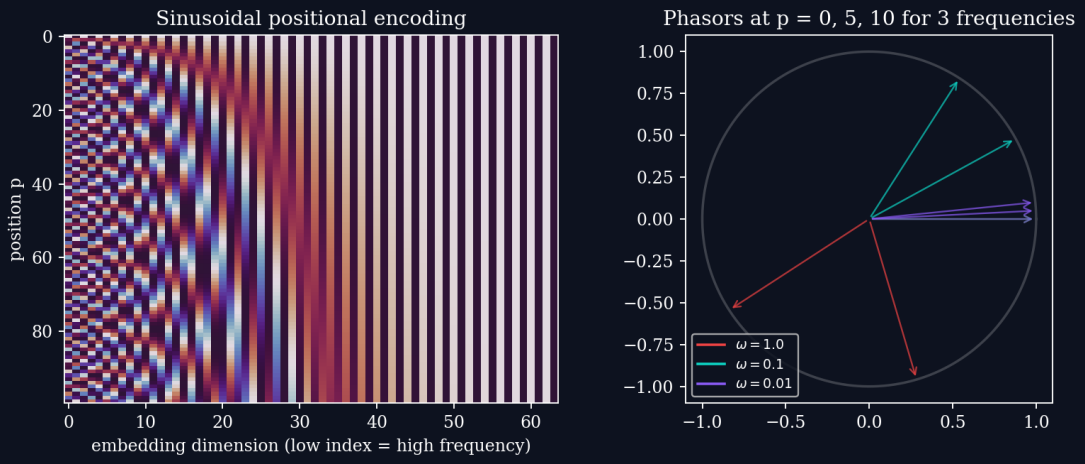


Figure 3: Left: sinusoidal positional encoding map ($d = 64$): fast-oscillating columns at low index, slow at high index. Right: phasors at positions 0, 5, 10 for three frequencies — high frequencies rotate quickly, low frequencies slowly.

---

9 9. MORE SIMILARITIES

DSP concept	ML concept	Comment
Matched filter / cross-correlation	Convolutional kernel as template detector; attention scores	Kernel responds maximally to the pattern it resembles; $\mathbf{q}^T\mathbf{k}$ is a correlation
Decimation + anti-alias filter	Strided conv/pooling; blurpool	Strided CNNs alias; low-pass before downsampling improves shift-invariance
Filter banks, wavelets, multirate	Multi-scale CNNs, U-Net, feature pyramids	Analysis/synthesis structure with skip links
STFT / spectrogram	Input features for audio networks, ViT patches	Time-frequency tiling
Linear prediction / AR model	Autoregressive language models	Predict next sample (token) from the past; LPC is a linear "language model"
Kalman filter (state estimation)	Recurrent state estimation; SSMs	Predict–correct recursion with gain like a gate
Parallel/feed-forward path $y = x + F(x)$	Residual connection	Direct path plus filtered path; improves gradient flow like a feed-forward comb
DC removal, AGC	Batch/Layer normalization	Remove mean, normalize power
Window functions, causal masks	Attention masks, causal attention	Restrict which samples contribute
Transform coding (DCT/JPEG)	Autoencoders, bottleneck codes	Compact representation, quantization = "discrete latents"
Compressed sensing, sparsity (L1)	Sparse coding, L1 regularization, pruning	Recovery from few measurements
Adaptive noise cancellation	Denoising networks, echo cancellers trained on reference	Reference correlated with noise, uncorrelated with signal
Lowpass smoothing, EMA	Momentum, EMA of weights, running stats	Section 4.3

10 10. LIMITS OF THE ANALOGIES (EXAM TRAPS & FACULTY INSIGHTS)

- **Linearity:** DSP filters (FIR/IIR) are linear and time-invariant; neural networks gain power from nonlinearities, so eigen/frequency analysis holds only for the linearized or linear versions.
- **Objectives differ:** LMS minimizes MSE with a convex quadratic bowl (unique minimum, guaranteed bound on μ); deep networks have non-convex losses.
- **RNN poles:** eigenvalues of $\mathbf{W}_h$ are poles only for the *linear* RNN; for tanh, use the Jacobian along the trajectory.
- **BM3D vs. attention:** similar in spirit (non-local similarity-weighted aggregation), different in algorithmic detail.
- **Positional encoding:** the claim is not that Transformers perform a Fourier transform, but that sinusoidal PE is a set of phasors whose shift property gives relative-position information; learned or RoPE positional schemes exist.

- **Cross-correlation vs. convolution:** CNN "convolution" is cross-correlation (no kernel flip).
- **SVD vs. Fourier denoising:** Fourier works when the signal is *band-limited*; SVD works when it is *low rank*. A sum of sinusoids is both.

---

11 11. QUICK REVISION CHEAT-SHEET

Idea	**Formula**	**ML name**
LMS	$\mathbf{w} \leftarrow \mathbf{w} + \mu e \mathbf{x}$	SGD / delta rule
Convergence along mode i	$(1 - \mu\lambda_i)^n$	Condition number / learning-rate limit
One-pole IIR	$y[n] = a\, y[n-1] + (1-a)x[n]$	EMA, momentum, forget gate
Linear RNN poles	$\mathrm{eig}(\mathbf{W}_h)$	Vanishing/exploding gradients
LTI eigenfunction	$e^{j\omega n} \to H(e^{j\omega})e^{j\omega n}$	Eigen-decomposition, PCA
Eckart–Young	$\min_{\mathrm{rank}\, k} \|\mathbf{A} - \mathbf{B}\| = \|\mathbf{A} - \mathbf{A}_k\|$	Low-rank denoising, LoRA
NLM weights	$\mathrm{softmax}(-\|\mathbf{p}_i - \mathbf{p}_j\|^2/h^2)$	Attention
Positional phasors	$e^{j\omega_i p},\ \omega_i = 10000^{-2i/d}$	Sinusoidal PE
Shift property	$PE(p+k) = \mathbf{R}(\omega k)PE(p)$	Relative position, RoPE